koroteev-test Добавлена методичка ML6.3 b3cd6a0 · last year History

Name	Last commit message	Last commit date
..		
img	Добавлена методичка ML6.3	last year
README.md	Добавлена методичка ML6.3	last year

README.md

Понижение размерности

Цель работы

Ознакомиться с основными методами понижения размерности, изучить их применение на практике, сравнить эффективность разных методов и визуализировать результаты.

Содержание работы

1. Сгенерируйте синтетический двумерный датасет с высокой степенью корреляции между признаками.
2. Понизьте размерность датасета до одной при помощи метода PCA и визуализируйте его до и после.
3. Примените метод главных компонент на более многомерных данных. Визуализируйте две главные компоненты.
4. Выберите оптимальное количество главных компонент по методу локтя.
5. Сгенерируйте двумерный датасет для классификации и примените на нем метод линейного дискриминантного анализа.
6. Сгенерируйте двумерный датасет для классификации и примените на нем метод понижения размерности t-SNE.
7. Сгенерируйте линейно неразделимый двумерный датасет для классификации и опробуйте на нем метод KernelPCA с различными ядрами.

Методические указания

В ходе выполнения этой работы мы познакомимся со разными методами понижения размерности. Эта задача в машинном обучении часто используется для визуализации многомерных данных, снижения объема данных, улучшения качества классификации, устранения мультиколлинеарности признаков. При этом, существуют разные методы понижения размерности, которые отличаются друг от друга не только по внутреннему устройству, но и по "техническим характеристикам", области применения, достоинствам и недостаткам. Поэтому по итогам выполнения данной работы нужно сформировать понимание того, в каких случаях какие методы следует применять.

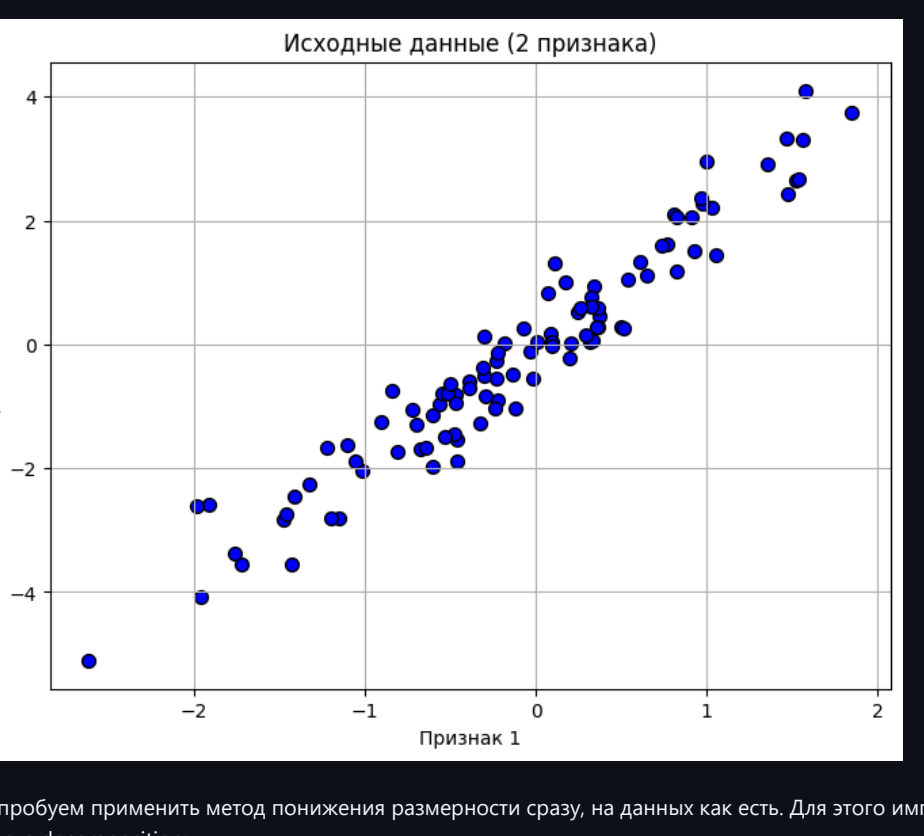
PCA на сгенерированных данных

Самый распространенный метод понижения размерности - метод главных компонент, PCA. Для того, чтобы понять смысл и основной механизм этого метода, мы разберем его действия на нескольких примерах. Для начала - на искусственных данных. Метод главных компонент выделяет самые информативные "направления" в пространстве признаков. Проще всего увидеть это направление на таком датасете, который демонстрирует сильную корреляцию.

Сгенерируем такой набор признаков, для наглядности, двумерный:

```
n_samples = 100
x = np.random.normal(0, 1, n_samples)  # Первый признак
y = 2 * x + np.random.normal(0, 0.5, n_samples)  # Второй признак (коррелирован с первым)

X = np.column_stack((x, y))
```

Можете сами варьировать численные параметры в этой генерации и смотреть, как это влияет на последующие результаты. В нашем случае датасет выглядит так:



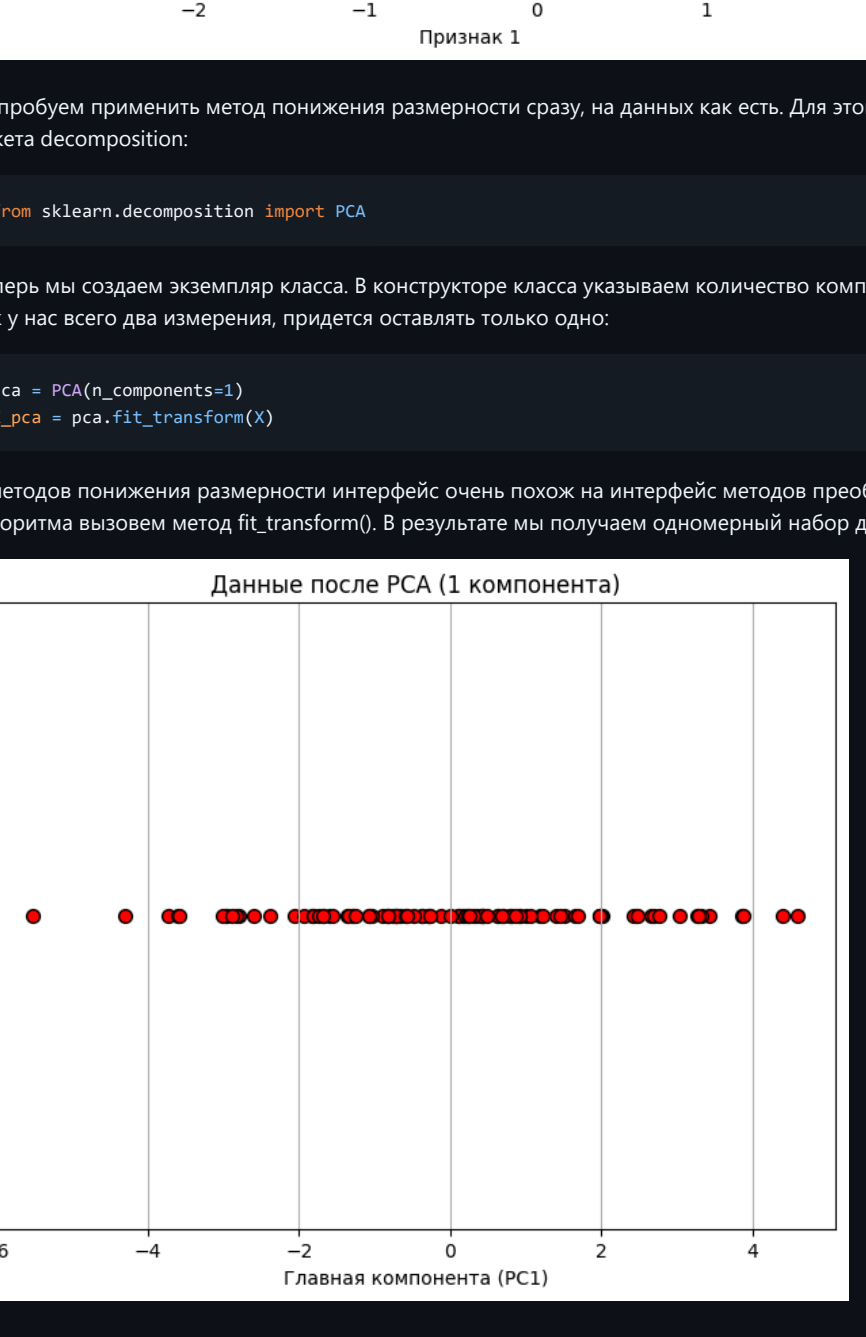
Попробуем применить метод понижения размерности сразу, на данных как есть. Для этого импортируем класс данного метода из пакета decomposition:

```
from sklearn.decomposition import PCA
```

Теперь мы создаем экземпляр класса. В конструкторе класса указываем количество компонент, которые хотим оставить в данных. Так как у нас всего два измерения, придется оставлять только одно:

```
pca = PCA(n_components=1)
X_pca = pca.fit_transform(X)
```

У методов понижения размерности интерфейс очень похож на интерфейс методов преобразования данных. Для применения алгоритма вызовем метод fit_transform(). В результате мы получаем одномерный набор данных, который выглядит вот так:



Это распределение, по сути, представляет собой проекцию исходного множества точек на определенную прямую. Эта прямая представляет собой направление, вдоль которого дисперсия исходного распределения максимальна. Этот вектор (направление) как раз и представляет собой главную компоненту.

Информацию о главной компоненте можно получить в соответствующих свойствах обученного объекта:

```
print(f"Главная компонента (направление): {pca.components_}")
print(f"Объясненная дисперсия: {pca.explained_variance_ratio_}")
```

Первое свойство хранит набор векторов главных компонент. Так как в нашем случае мы оставили только одно измерение, и главная компонента тоже будет одна. Это двумерный вектор, который и представляет собой направление в исходном пространстве.

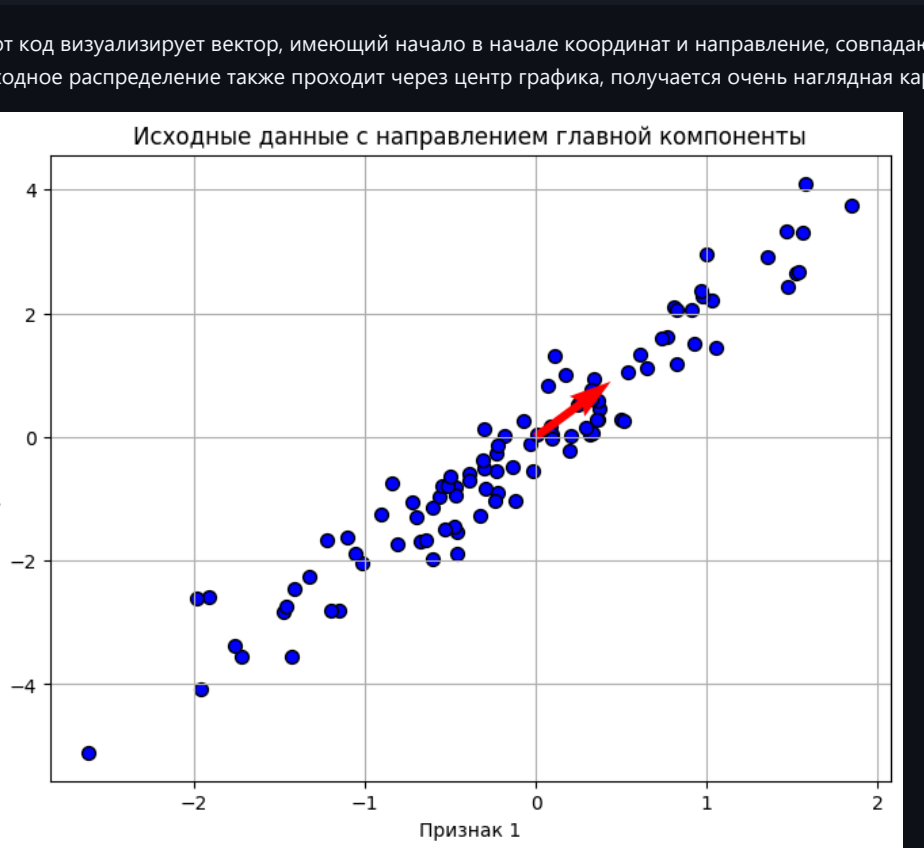
Второй список хранит доли объясненной дисперсии по главным компонентам. Тут у нас опять только одно значение. Оно значит, что понижение размерности с двух до одного измерения сохраняет 98% исходной дисперсии выборки.

```
Главная компонента (направление): [[0.44011603 0.89794091]]
Объясненная дисперсия: [0.98901026]
```

В нашем случае двумерных данных можно очень наглядно визуализировать главную компоненту в исходном распределении. Можно изобразить вектор этого направления на диаграмме рассеяния:

```
plt.scatter(X[:, 0], X[:, 1], c='blue', edgecolor='k', s=50)
pc1_direction = pca.components_[0]
plt.quiver(0, 0, pc1_direction[0], pc1_direction[1], angles='xy', scale_units='xy', scale=1, color='red', width=0.01)
```

Этот код визуализирует вектор, имеющий начало в начале координат и направление, совпадающее с главной компонентой. Так как исходное распределение также проходит через центр графика, получается очень наглядная картина:



Попробуйте самостоятельно изменить парамеры исходного распределения и посмотреть, как это влияет на направление главной компоненты.

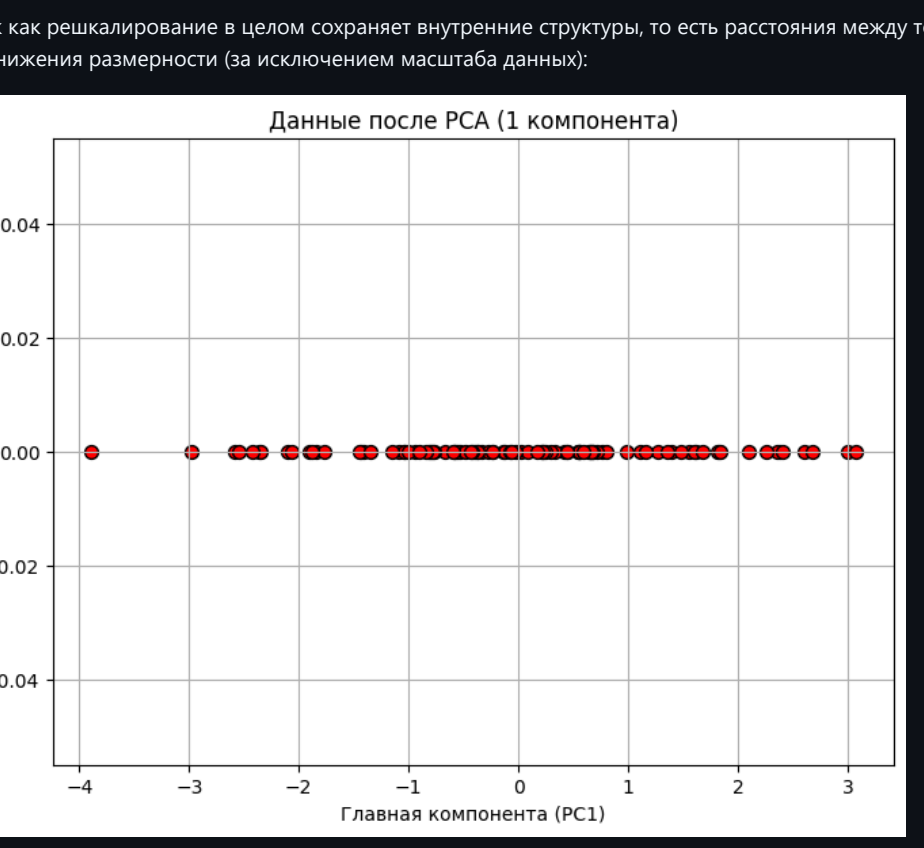
На практике, данные перед применение метода понижения размерности необходимо нормировать. Посмотрим, как нормировка данных влияет на результат. Воспользуемся уже известным методом стандартизации:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Теперь воспользуемся тем же методом понижения размерности, но уже обучим его на масштабированных данных:

```
pca = PCA(n_components=1)
X_pca = pca.fit_transform(X_scaled)

plt.scatter(X_pca, np.zeros_like(X_pca), c='red', edgecolor='k', s=50)
```

Так как решкалирование в целом сохраняет внутренние структуры, то есть расстояния между точками, это не влияет на результат понижения размерности (за исключением масштаба данных):



PCA на многомерных данных

по аналогии проследим работу того же алгоритма, но уже на данных, состоящих из большего количества измерений. Для примера возьмем встроенный датасет о диагностике рака:

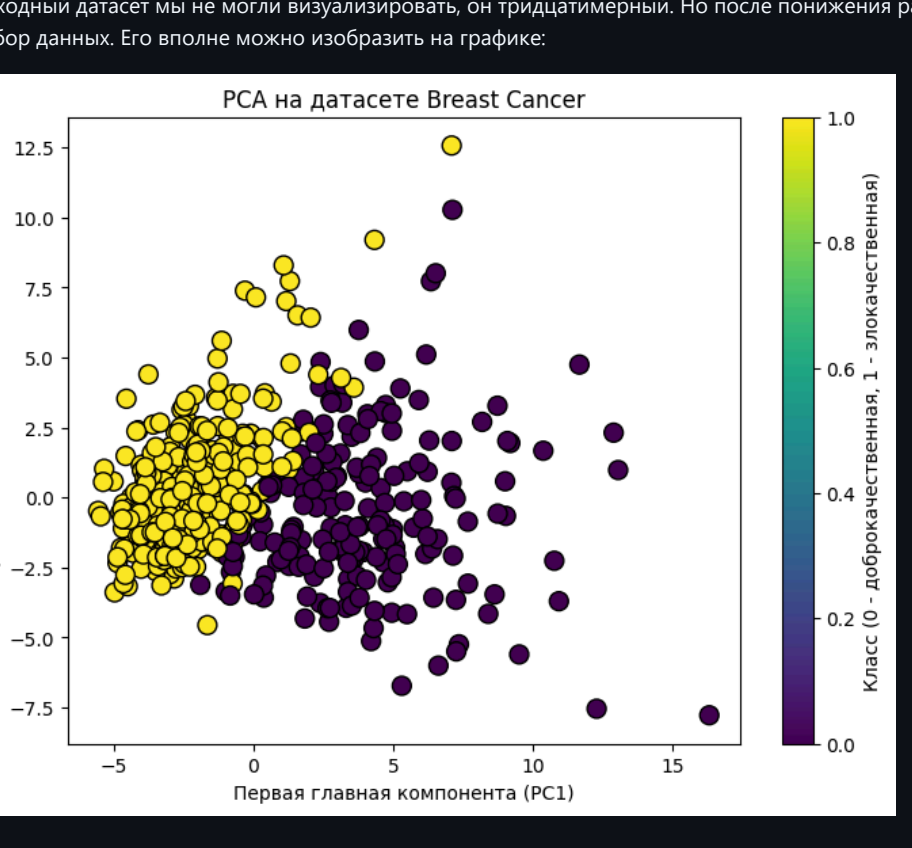
```
from sklearn.datasets import load_breast_cancer

data = load_breast_cancer()
X = data.data
y = data.target
X.shape
```

Этот набор данных содержит 30 столбцов. Это хорошее число, имеет смысл уменьшить. Пока запомним, что в исходных данных именно 30 столбцов, это пригодится позднее.

Теперь применим метод главных компонент. Сейчас выберем уже две главные компоненты. В таком случае результат можно будет визуализировать. Обратите внимание, что мы обязательно применяем нормализацию:

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

pca = PCA(n_components=2)  # Уменьшаем до 2 компонент
X_pca = pca.fit_transform(X_scaled)
```

Исходный датасет мы не могли визуализировать, он тридцатимерный. Но после понижения размерности у нас получился двумерный набор данных. Его вполне можно изобразить на графике:



Если вы задаетесь вопросом, что именно отложено по осям, то ответ может вас разочаровать - главные компоненты. Никакого предметного смысла они не несут.

Мы здесь видим, по сути, проекцию набора точек из тридцатимерного пространства на плоскость. Причем эта плоскость расположена внутри этого тридцатимерного пространства таким образом, чтобы сохранить максимальную дисперсию. Другими словами, эта плоскость расположена так, чтобы в проекции точки были максимально разбросаны.

Можно задаться вопросом, как эта плоскость расположена внутри этого тридцатимерного пространства. Плоскость задается двумя векторами. Именно эти вектора и есть главные компоненты - направления, вдоль которых дисперсия максимальна.

```
print(f"Главная компонента (направление): {pca.components_}")
```

Конечно, два тридцатимерных вектора не очень информативны. Но вот они:

```
Главная компонента (направление): [[ 0.21890244  0.10372458  0.22753729  0.22099499  0.14258969  0.23928535
   0.25840048  0.26085376  0.13816696  0.06436335  0.20597878  0.01742803
   0.21132592  0.20286964  0.01453145  0.17039345  0.15358979  0.1834174
   0.04249842  0.10256832  0.22799663  0.10446933  0.23663968  0.22487053
   0.12795256  0.21009588  0.22876753  0.25088597  0.12290456  0.13178394]
 [-0.23385713 -0.05970609 -0.21518136 -0.23107671  0.18611302  0.15189161
   0.06016536 -0.0347675   0.19034877  0.36657547 -0.10555215  0.08997968
  -0.08945723 -0.15229263  0.20443045  0.2327159   0.19720728  0.13032156
   0.183848    0.28009203 -0.21986638 -0.0454673  -0.19987843 -0.21935186
   0.17230435  0.14359317  0.09796411 -0.00825724  0.14188335  0.27533947]]
```

В отличие от предыдущего примера, мы все-таки очень сильно сократили размерность. Это значит, что точки все равно сильно сместились относительно своего изначального положения в проекцию. То есть часть данных мы потеряли. Давайте посмотрим, какую именно часть выборочной дисперсии мы сохранили после понижения размерности:

```
explained_variance = pca.explained_variance_ratio_
print(f"Объясненная дисперсия каждой компоненты: {explained_variance}")
print(f"Суммарная объясненная дисперсия: {sum(explained_variance):.2f}")
```

В соответствующем свойстве объекта модели лежит массив, который показывает долю сохраненной (объясненной) дисперсии данных по каждой компоненте. Здесь мы видим, что проекция на первую главную компоненту сохраняет 44% дисперсии исходных данных. Это значит, что если бы мы сократили размерность до одной, то потеряли бы 56% исходной информации.

```
Объясненная дисперсия каждой компоненты: [0.44272026 0.18971182]
Суммарная объясненная дисперсия: 0.63
```

Введение второй компоненты добавляет еще 19% дисперсии. Таким образом, всего у нас получается 63%. Можно считать, что понижение размерности в данном случае приводит к потере 37% информации о разбросе точек, имеющейся в исходном наборе.

Метод локтя для PCA

Метод понижения размерности полезен для визуализации данных, но применяют его не только для этого. Он может быть хорошим помощником в тех случаях, когда в исходных данных слишком много признаков, их число надо сократить, но так, чтобы максимально сохранить имеющуюся информацию. В таком случае встает вопрос выбора количества компонент.

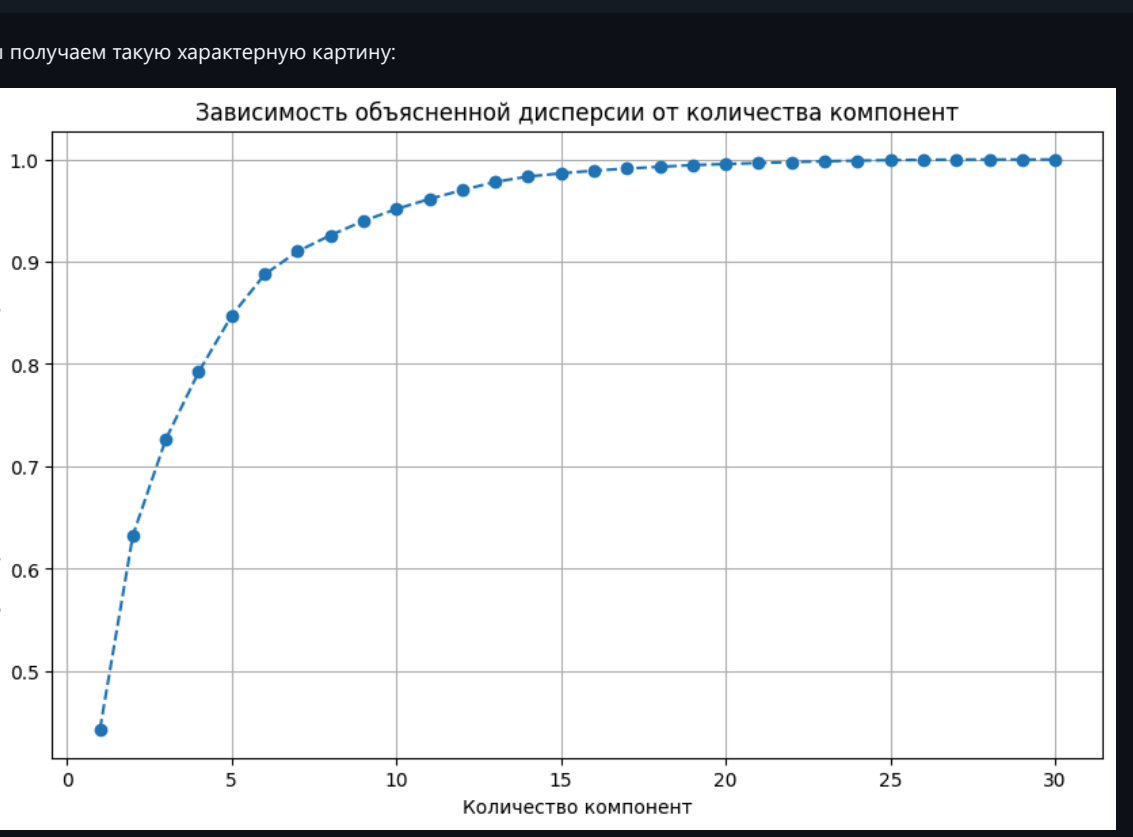
При определении оптимального количества компонент следует ориентироваться на то, насколько быстро растет количество информации в зависимости от количества измерений. Для того, чтобы отследить эту зависимость, построим график. Для этого построим алгоритм главных компонент с количеством компонент от 1 до 30:

```
n_components_range = range(1, 31)
explained_variance = []
```

Для каждого количества обучим и оценим соответствующую модель и запомним процент объясненной дисперсии:

```
for n in n_components_range:
    pca = PCA(n_components=n)
    pca.fit(X_scaled)
    explained_variance.append(sum(pca.explained_variance_ratio_))
```

Мы получаем такую характерную картину:



Единственная главная компонента обеспечивает 44% вариации исходных данных (мы уже видели это число ранее). При добавлении второй - мы получаем значительную прибавку. При добавлении третьей - прибавка чуть меньше. С каждой новой компонентой мы получаем чуть меньше дополнительной информации.

Смысл в том, чтобы найти такое количество, которое дает оптимальное количество сохраненной информации. Для этого найдем такое значение, в котором данная кривая максимально изгибается. Это и будет локоть. Можно предположить, что в данном случае оптимальное количество 6 или 7 компонент. Если брать больше, дополнительные измерения уже не обеспечат такого значительного прироста информации.

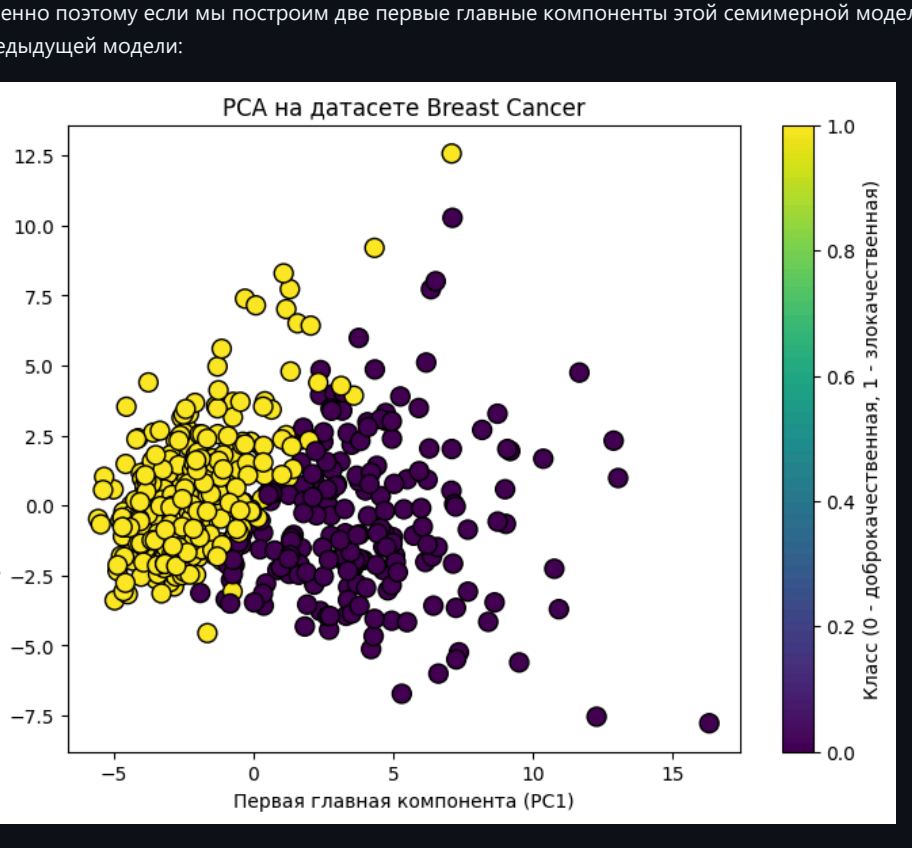
```
pca = PCA(n_components=7)  # Уменьшаем до 7 компонент
X_pca = pca.fit_transform(X_scaled)

# Объясненная дисперсия
explained_variance = pca.explained_variance_ratio_
print(f"Объясненная дисперсия каждой компоненты: {explained_variance}")
print(f"Суммарная объясненная дисперсия: {sum(explained_variance):.2f}")
```

При выводе массива значений объясненной дисперсии мы видим два уже знакомых нам значения, а числа дальше показывают, сколько дополнительной объясненной дисперсии прибавляет каждая следующая главная компонента:

```
Объясненная дисперсия каждой компоненты: [0.44272026 0.18971182 0.09393163 0.06602135 0.05495768 0.04024522
 0.02250734]
Суммарная объясненная дисперсия: 0.91
```

Обратите внимание, что в данном методе первые компоненты не зависят от того, сколько компонент мы задала. Другими словами, метод всегда найдет одну и ту же главную компоненту, ее положение не зависит от количества компонент, которые мы оставляем. То есть, задание количества компонент по сути является лишь критерием остановки алгоритма (как ограничение глубины решающего дерева).

Именно поэтому если мы построим две первые главные компоненты этой семимерной модели, они будут полностью идентичны предыдущей модели:



Это, в частности значит, что для выбора количества компонент методом локтя необязательно было обучать тридцать моделей подряд. Попробуйте самостоятельно достичь того же результата, но гораздо более рациональным способом.

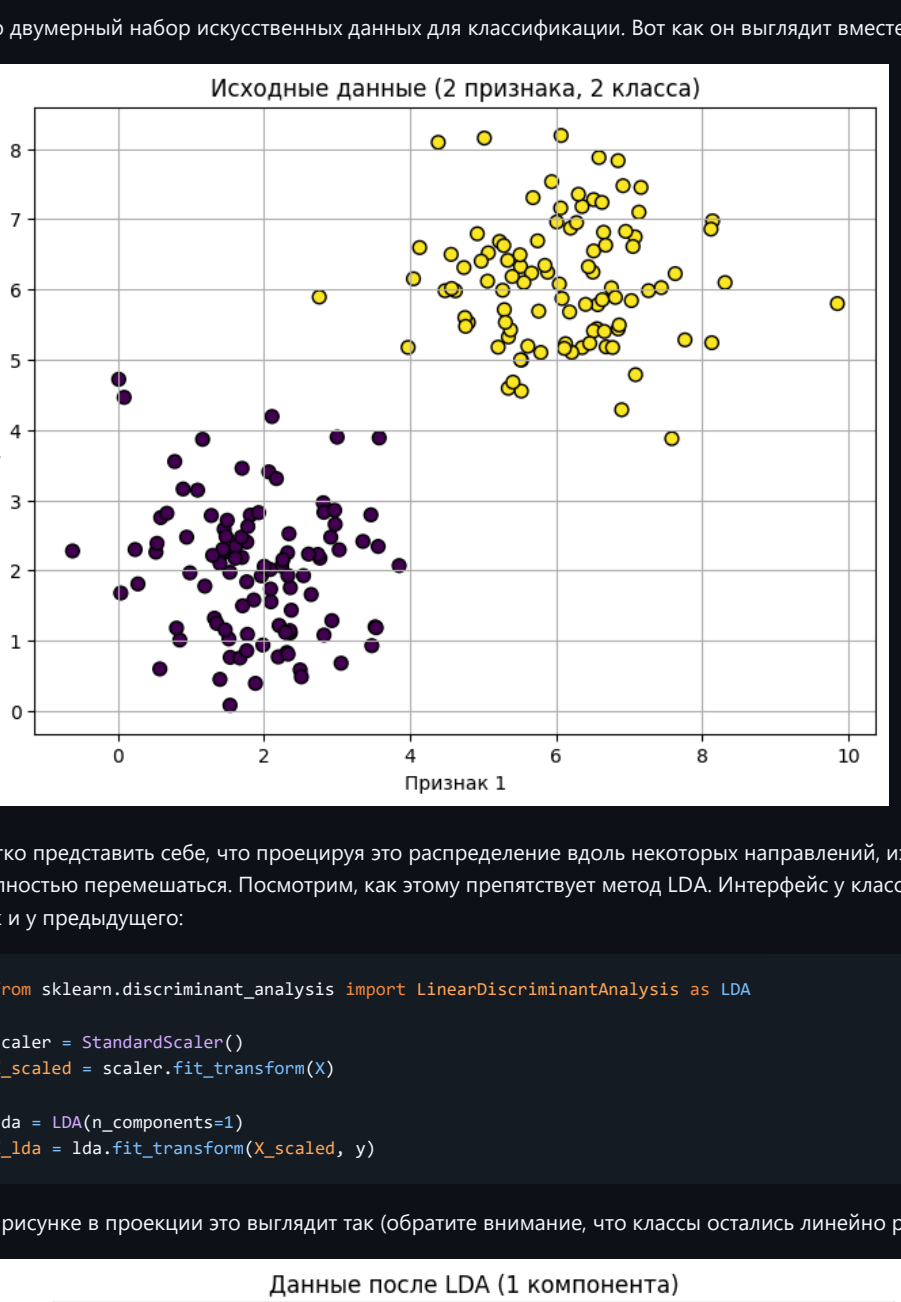
Метод LDA

Метод главных компонент неплохо работает в большинстве случаев. Однако, у него есть один недостаток - он принимает во внимание только взаимное расположение точек. Если перед вами датасет для классификации, этот метод может найти такие компоненты, то есть такие направления в исходном пространстве, после проекции на которые, точки разных классов могут оказаться еще ближе друг к другу, чем были в исходном распределении. Это может сильно осложнить задачу классификации, когда после проекции точки классов перемешаются.

Именно для решения этой проблемы существует метод линейного дискриминантного анализа. Он принимает во внимание значение целевой переменной. И выбирает направления проекции таким образом, чтобы максимизировать расстояние между классами. Рассмотрим, например, такой датасет:

```
n_samples = 100

x1 = np.random.normal(2, 1, n_samples)
y1 = np.random.normal(2, 1, n_samples)
x2 = np.random.normal(6, 1, n_samples)
y2 = np.random.normal(6, 1, n_samples)

X = np.vstack((np.column_stack((x1, y1)), np.column_stack((x2, y2))))
y = np.hstack((np.zeros(n_samples), np.ones(n_samples)))
```

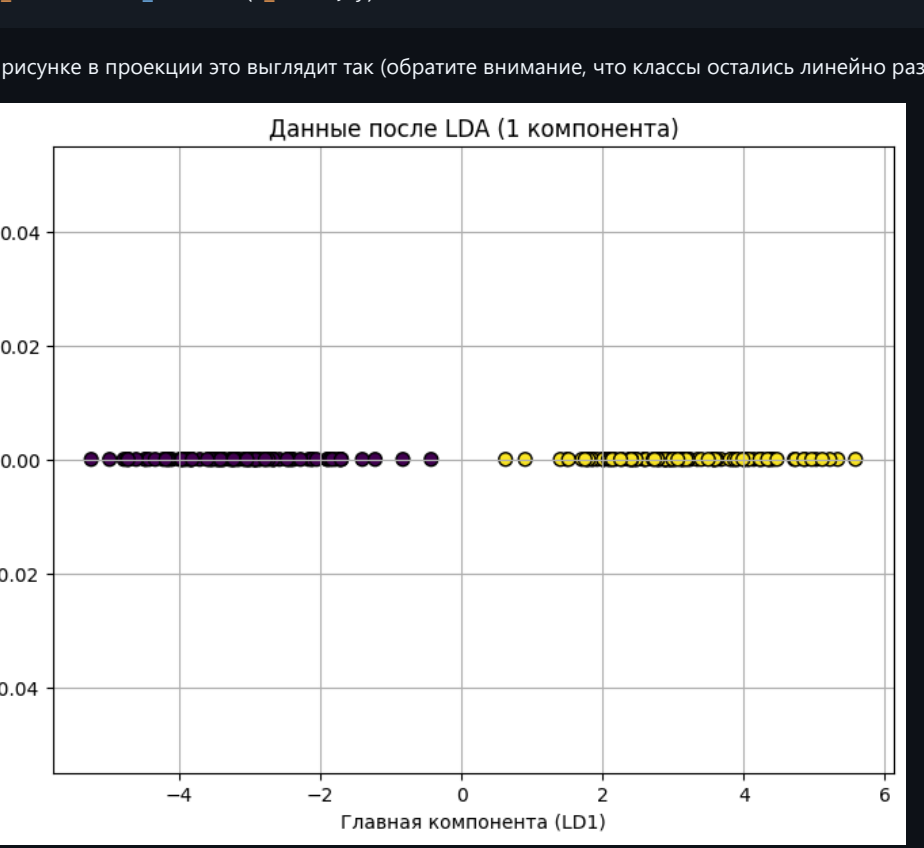
Это двумерный набор искусственных данных для классификации. Вот как он выглядит вместе с информацией о классах:



Легко представить себе, что проецируя это распределение вдоль некоторых направлений, изначально очень разделимые классы могут полностью перемешаться. Посмотрим, как этому препятствует метод LDA. Интерфейс у класса этого метода, конечно, точно такой же, как и у предыдущего:

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

lda = LDA(n_components=1)
X_lda = lda.fit_transform(X_scaled, y)
```

На рисунке в проекции это выглядит так (обратите внимание, что классы остались линейно разделимыми):



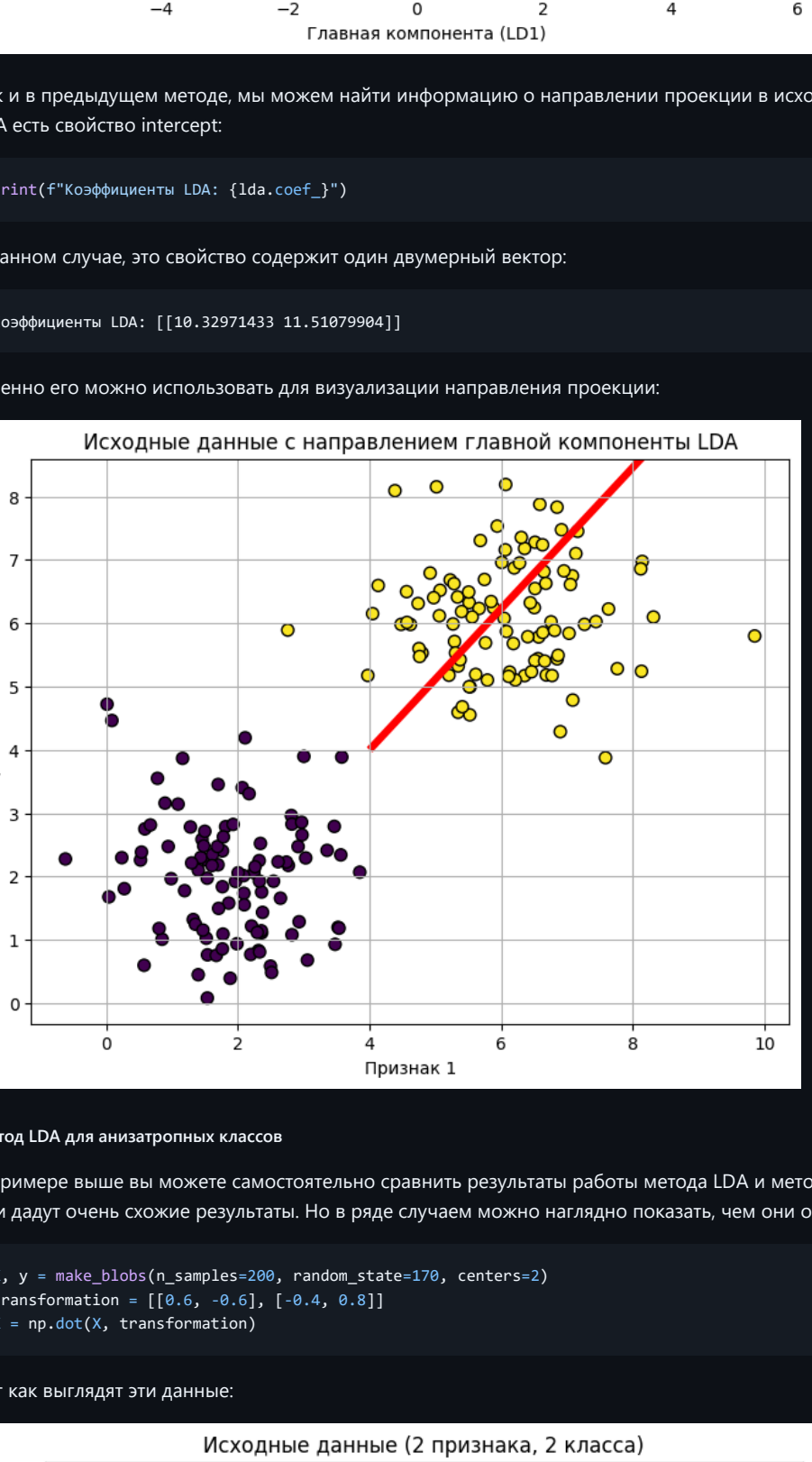
Как и в предыдущем методе, мы можем найти информацию о направлении проекции в исходном пространстве. Для этого в методе LDA есть свойство intercept:

```
print(f"Коэффициенты LDA: {lda.coef_}")
```

В данном случае, это свойство содержит один двумерный вектор:

```
Коэффициенты LDA: [[10.32971433 11.51079904]]
```

Именно его можно использовать для визуализации направления проекции:

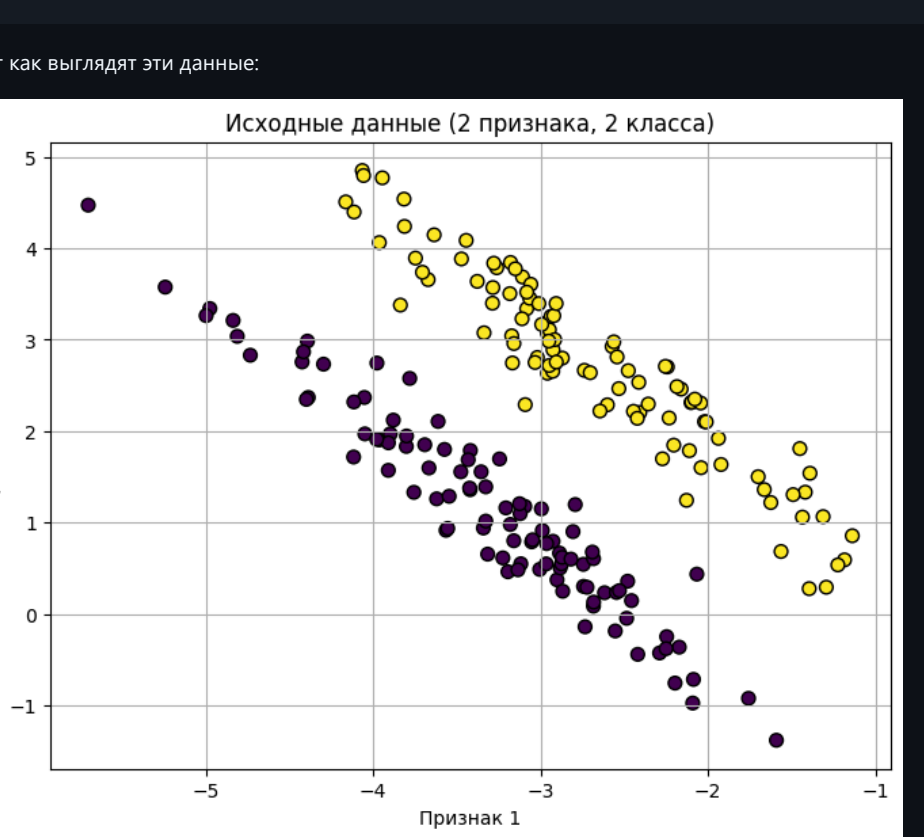


Метод LDA для анизатропных классов

В примере выше вы можете самостоятельно сравнить результаты работы метода LDA и метода главных компонент. В данном случае они дадут очень схожие результаты. Но в ряде случаем можно наглядно показать, чем они отличаются. Рассмотри такой датасет:

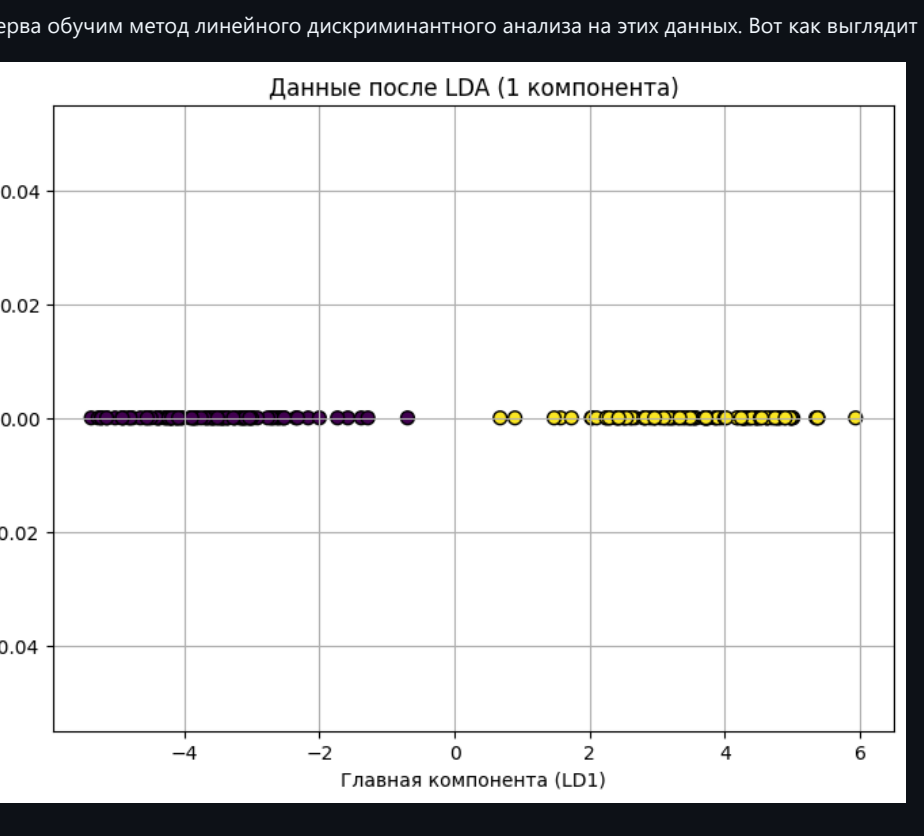
```
X, y = make_blobs(n_samples=200, random_state=170, centers=2)
transformation = [[0.6, -0.6], [-0.4, 0.8]]
X = np.dot(X, transformation)
```

Вот как выглядят эти данные:

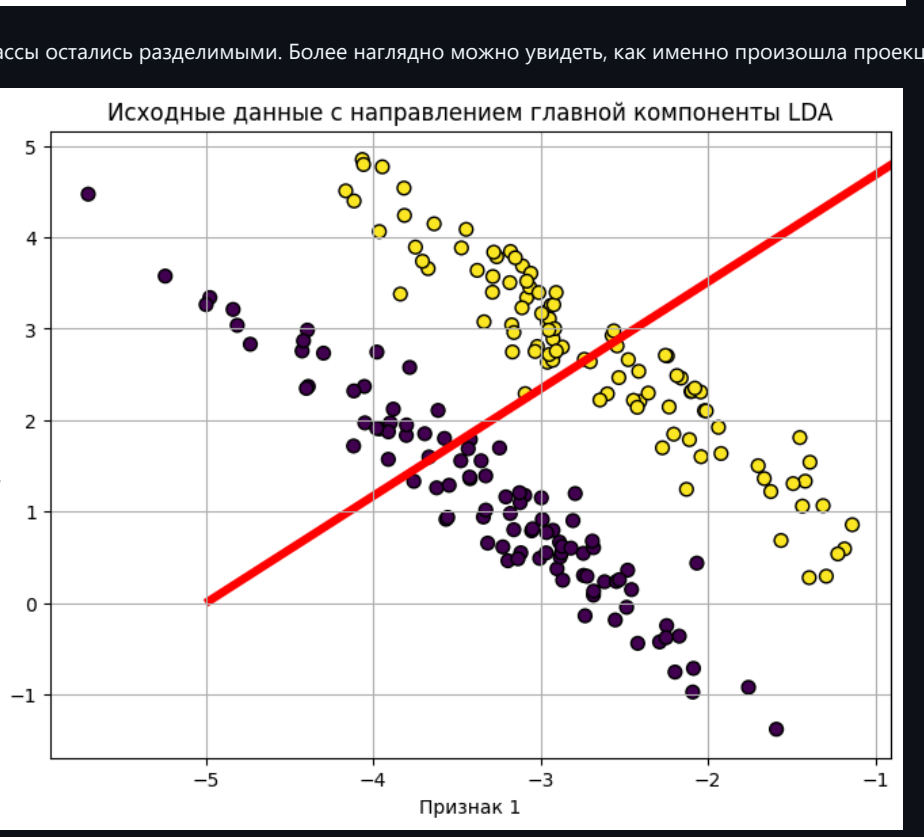


Легко заметить, что направление максимальной дисперсии в данном случае приведет к полному перемешиванию классов в данной выборке. Мы увидим это на практике в сравнении двух методов.

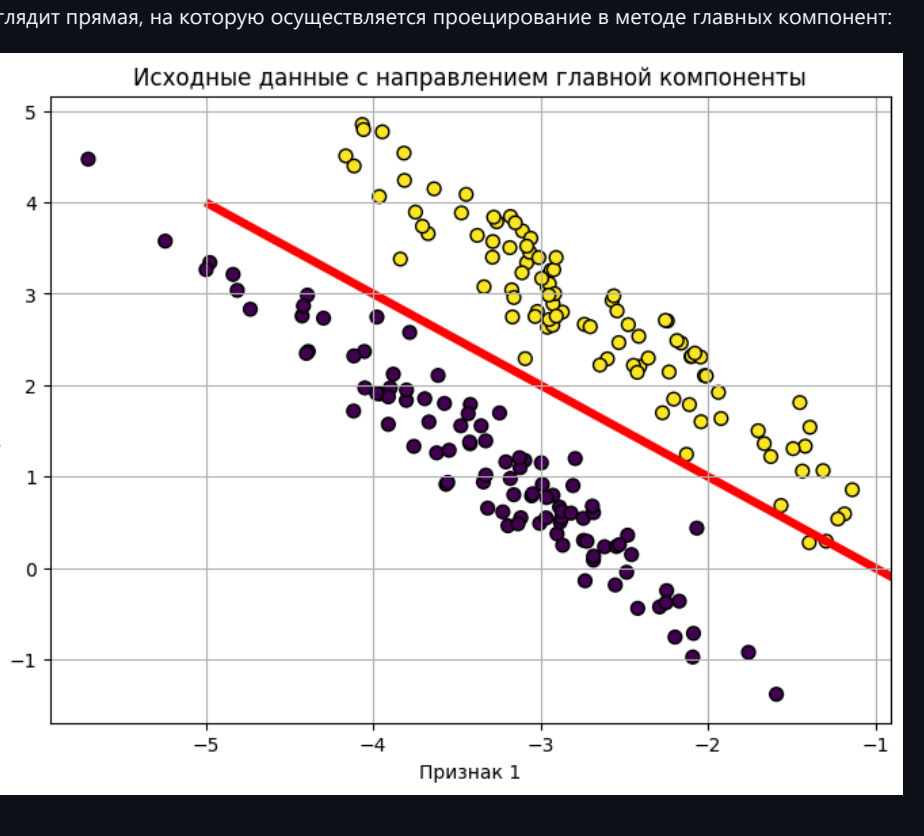
Сперва обучим метод линейного дискриминантного анализа на этих данных. Вот как выглядит результат понижения размерности:



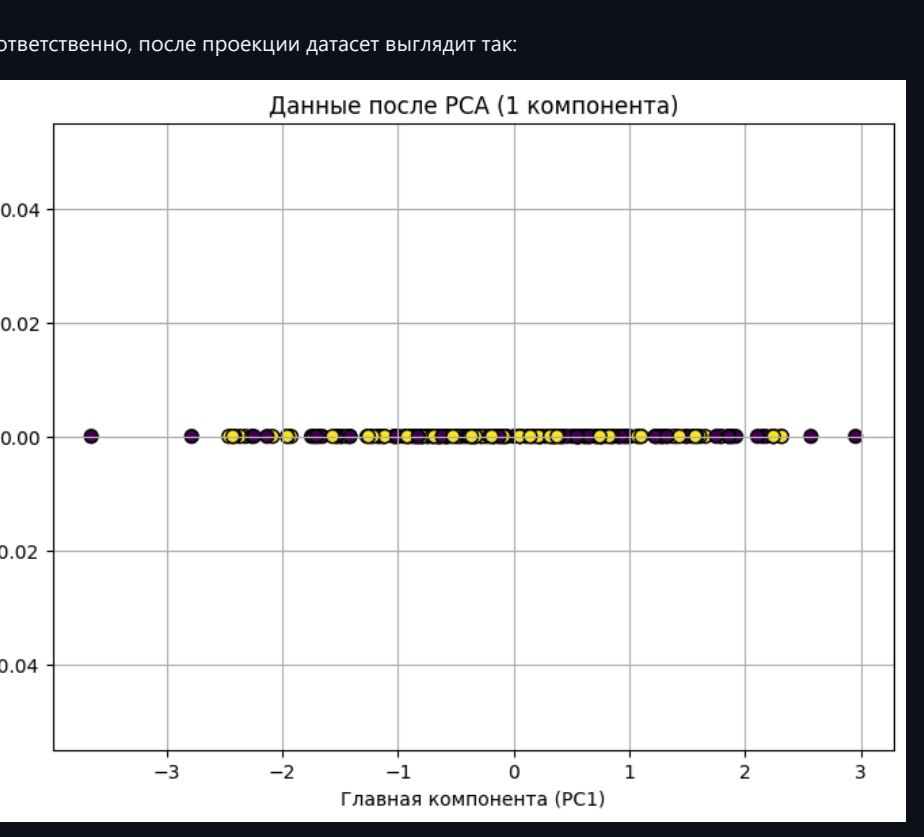
Классы остались разделимыми. Более наглядно можно увидеть, как именно произошла проекция в исходном пространстве:



Алгоритм выбрал именно то направление, которое сохраняет разделимость классов. Примечательно то, что метод главных компонент не принимает во внимание значение целевой переменной и, как следствие, приводит к совершенно другой проекции. Вот как выглядит прямая, на которую осуществляется проецирование в методе главных компонент:



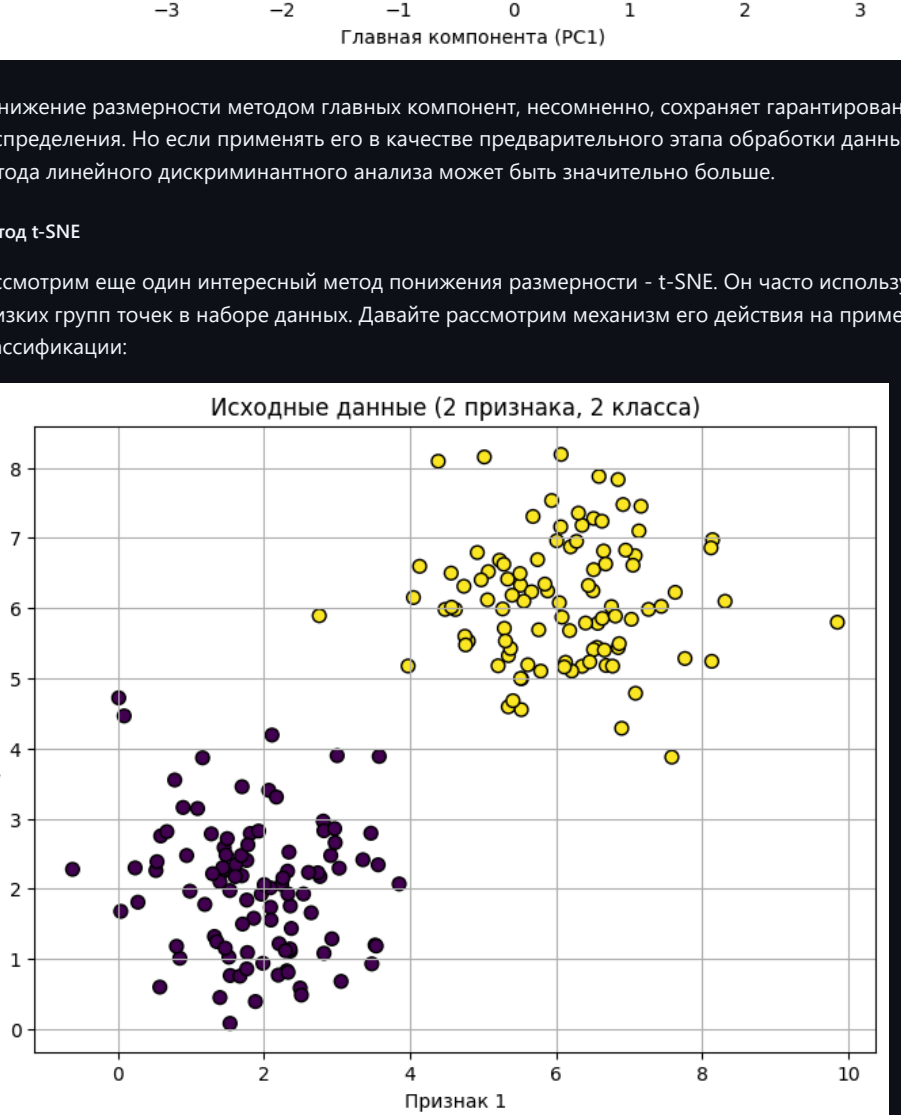
Соответственно, после проекции датасет выглядит так:



Понижение размерности методом главных компонент, несомненно, сохраняет гарантированно больше информации от исходного распределения. Но если применять его в качестве предварительного этапа обработки данных перед классификацией, то пользы от метода линейного дискриминантного анализа может быть значительно больше.

Метод t-SNE

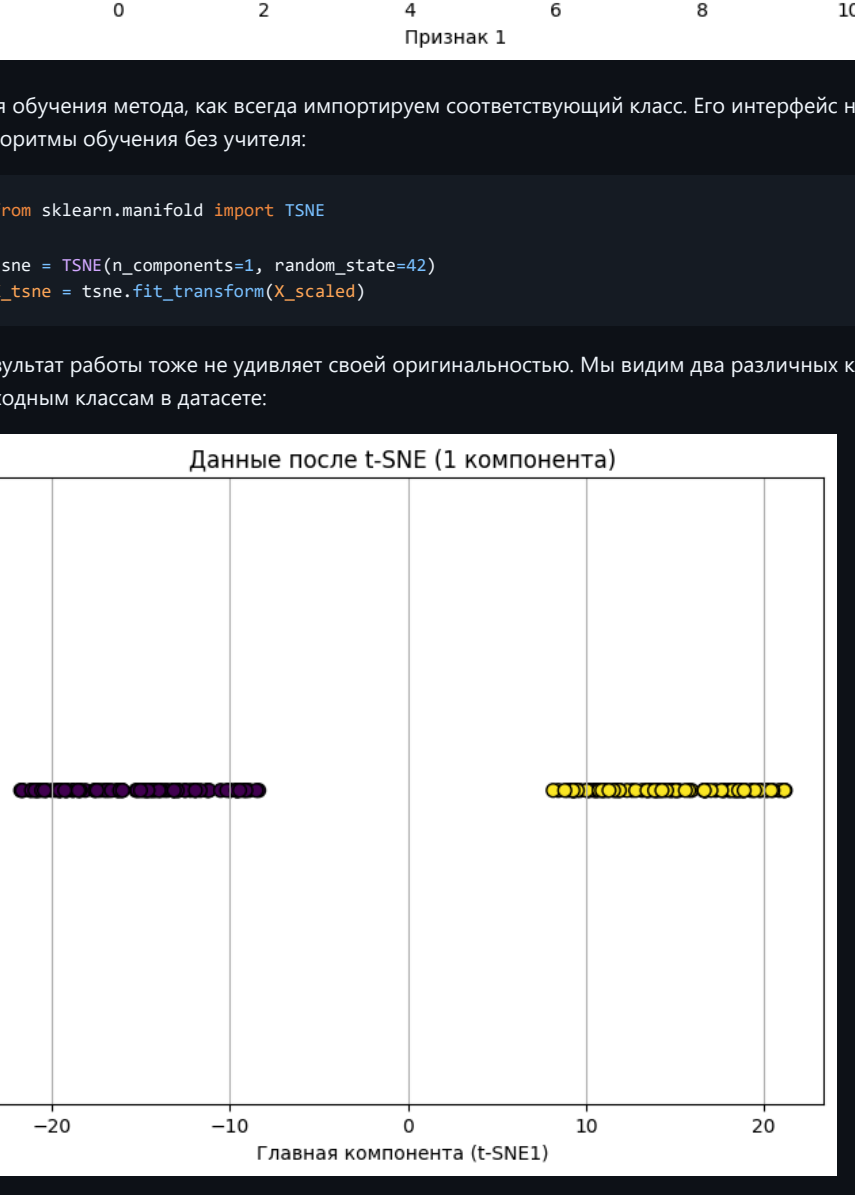
Рассмотрим еще один интересный метод понижения размерности - t-SNE. Он часто используется для визуализации кластеров - то есть близких групп точек в наборе данных. Давайте рассмотрим механизм его действия на примере уже использованного нами датасета для классификации:



Для обучения метода, как всегда импортируем соответствующий класс. Его интерфейс не отличается от других классов, реализующих алгоритмы обучения без учителя:

```
from sklearn.manifold import TSNE

tsne = TSNE(n_components=1, random_state=42)
X_tsne = tsne.fit_transform(X_scaled)
```

Результат работы тоже не удивляет своей оригинальностью. Мы видим два различных кластера точек, которые соответствуют двум исходным классам в датасете:



Однако, имейте в виду, что данному алгоритму нигде не передавались метки классов, то есть значение целевой переменной. Алгоритм выделил эти группы точек самостоятельно, ориентируясь исключительно на их схожесть.

Самостоятельно проверьте работу этого алгоритма на созданном в предыдущем пункте анизотропном датасете.

Алгоритм t-SNE является параметрическим. У него есть один главный гиперпараметр, который существенно влияет на работу алгоритма. Это так называемый perplexity - он влиет на то, какие структуры будут превалировать в анализе, крупномасштабные или мелкие.

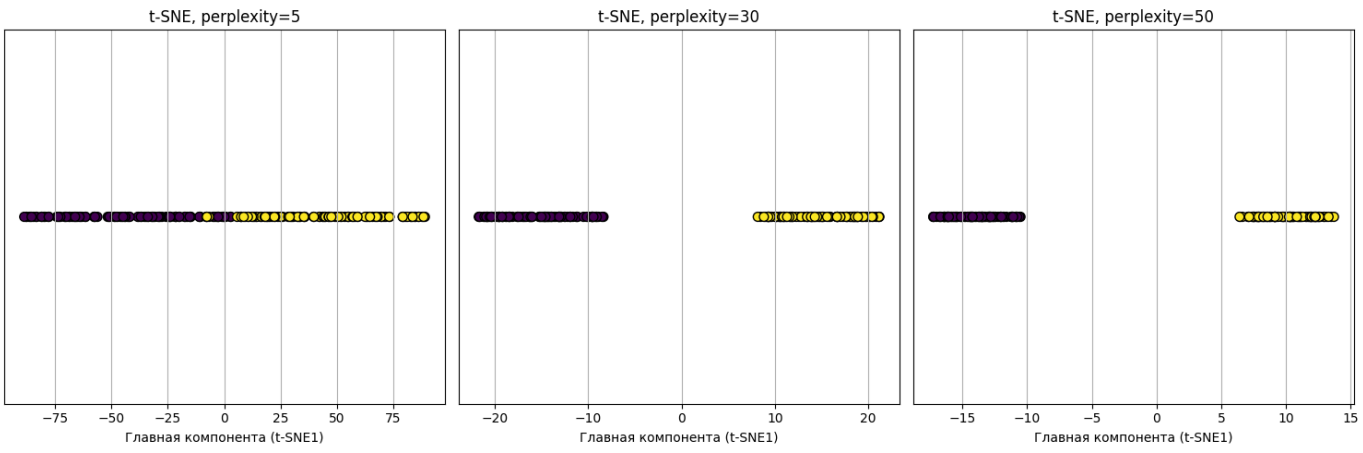
Теоретически очень сложно прочувствовать влияние этого параметр на суть работы алгоритма. Поэтому мы попробуем разные значения на практике. Авторы алгоритма рекомендуют в качестве рабочего диапазона значений этого параметра разброс от 5 до 50. Попробуем несколько конкретных значений из этого диапазона:

```
perplexity_values = [5, 30, 50]
plt.figure(figsize=(15, 5))
```

Теперь обучим для каждого значения соответствующий алгоритм и визуализируем его результаты:

```
for i, perplexity in enumerate(perplexity_values):
    tsne = TSNE(n_components=1, perplexity=perplexity, random_state=42)
    X_tsne = tsne.fit_transform(X_scaled)

    plt.subplot(1, 3, i+1)
    plt.scatter(X_tsne, np.zeros_like(X_tsne), c=y, cmap='viridis', edgecolor='k', s=50)
    plt.title(f't-SNE, perplexity={perplexity}')
    plt.xlabel('Главная компонента (t-SNE1)')
    plt.yticks([])
    plt.grid(True)
```

Мы видим разное формирование кластеров точек в зависимости от значения preplexity.



Конечно, в таком одномерном варианте сложно оценить формирование структур в данных. Поэтому самостоятельно проверьте работу этого алгоритма на более многомерных данных.

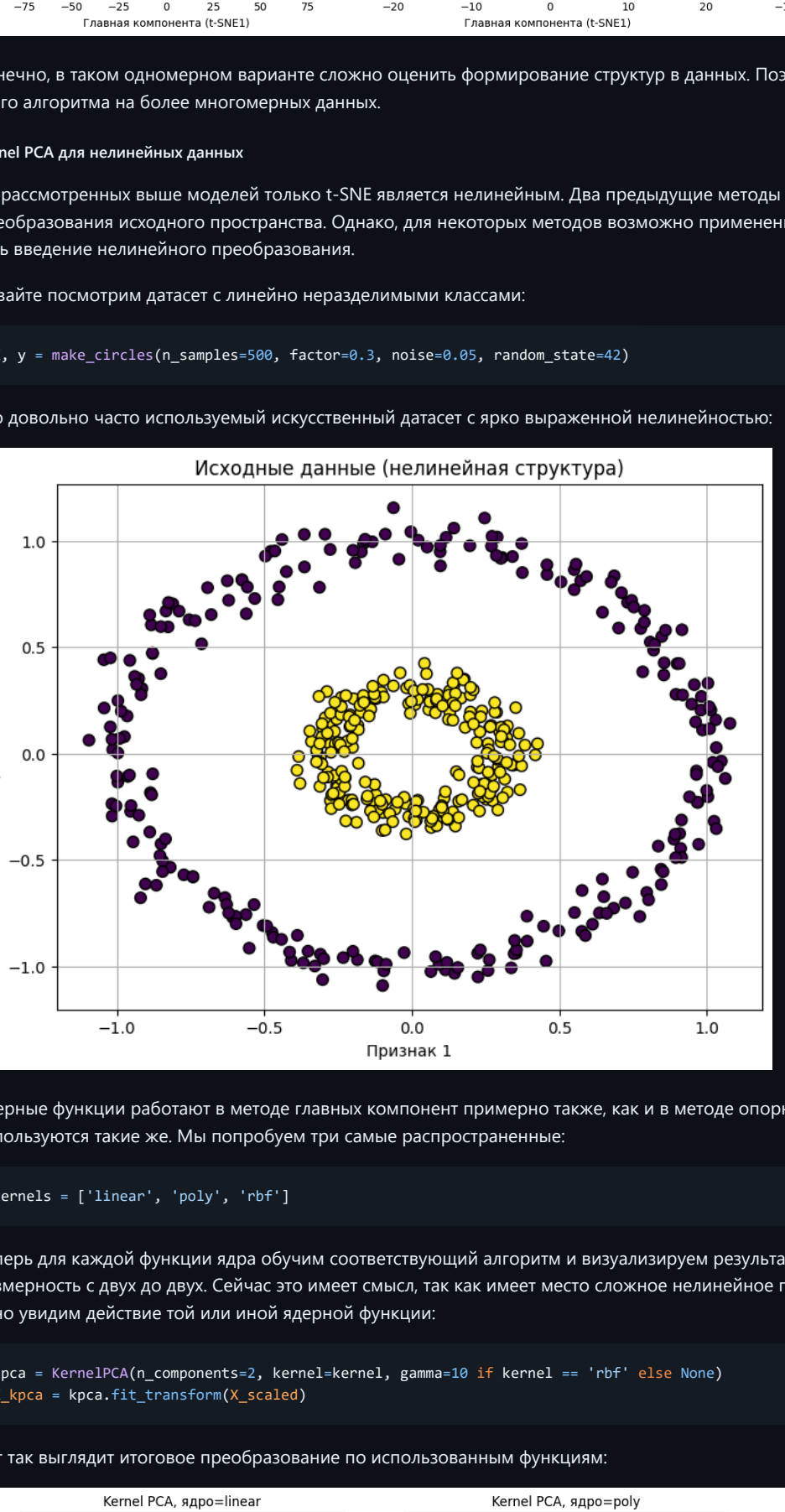
Kernel PCA для нелинейных данных

Из рассмотренных выше моделей только t-SNE является нелинейным. Два предыдущие методы осуществляют только линейные преобразования исходного пространства. Однако, для некоторых методов возможно применение трюка с ядерными функциями, то есть введение нелинейного преобразования.

Давайте посмотрим датасет с линейно неразделимыми классами:

```
X, y = make_circles(n_samples=500, factor=0.3, noise=0.05, random_state=42)
```

Это довольно часто используемый искусственный датасет с ярко выраженной нелинейностью:



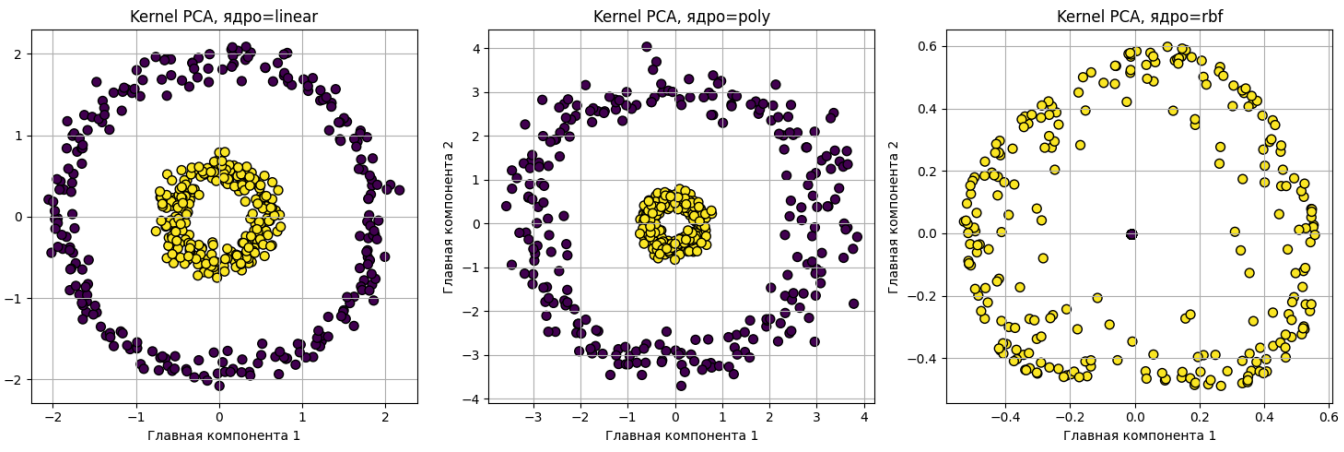
Ядерные функции работают в методе главных компонент примерно также, как и в методе опорных векторов. И ядерные функции используются такие же. Мы попробуем три самые распространенные:

```
kernels = ['linear', 'poly', 'rbf']
```

Теперь для каждой функции ядра обучим соответствующий алгоритм и визуализируем результаты. Для разнообразия мы "понизим" размерность с двух до двух. Сейчас это имеет смысл, так как имеет место сложное нелинейное преобразование, так что после него мы явно увидим действие той или иной ядерной функции:

```
kpca = KernelPCA(n_components=2, kernel=kernel, gamma=10 if kernel == 'rbf' else None)
X_kpca = kpca.fit_transform(X_scaled)
```

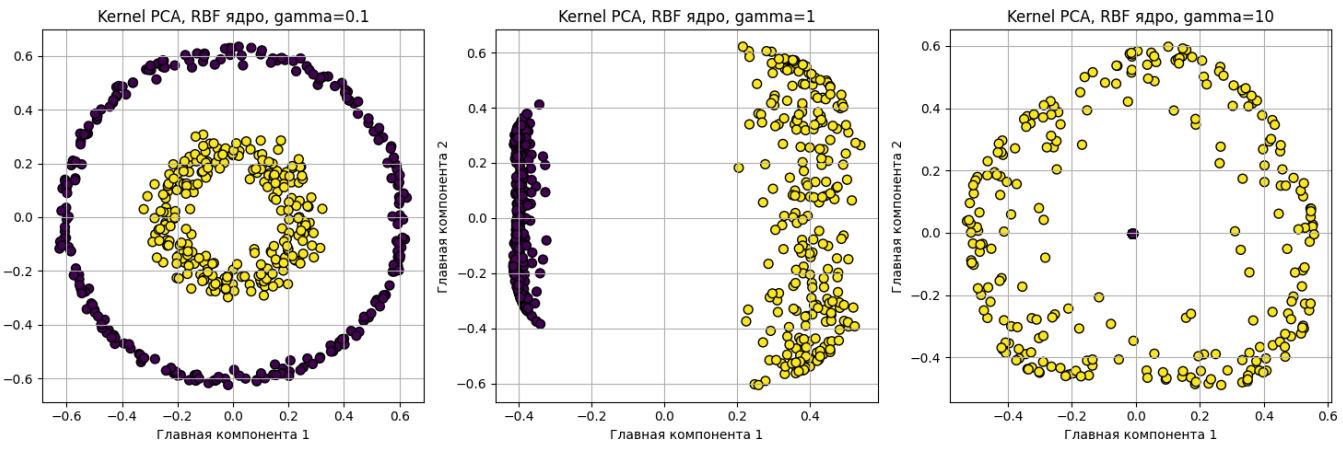
Вот так выглядит итоговое преобразование по использованным функциям:



Кроме применения разных функций, можно вспомнить, что радиально-базисная ядерная функция параметризуется степенью гамма. Точно также эмпирически можно проверить разные значения этого гиперпараметра:

```
gamma_values = [0.1, 1, 10]
```

Вот как выглядят получившиеся распределения:



Самостоятельно попробуйте другие значения этого гиперпараметра, промежуточные, для того, чтобы прочувствовать его влияние на результат работы алгоритма.

Задания для самостоятельного выполнения

1. Попробуйте изменить уровень шума в данных (например, увеличить или уменьшить шум в y) и посмотрите, как это влияет на результат PCA.
2. Добавьте третий признак, который также коррелирует с первыми двумя, и примените PCA с n_components=2.
3. Попробуйте изменить расположение классов (например, сделать их ближе друг к другу) и посмотрите, как это влияет на результат LDA.
4. Добавьте третий класс и примените LDA с n_components=2.
5. Сравните LDA с PCA на этих же данных. Какой метод лучше разделяет классы?
6. Попробуйте изменить параметр degree для полиномиального ядра метода KernelPCA и посмотрите, как это влияет на результат.
7. Примените все три изученных в этой работе метода к датасету для классификации по вашему выбору.
8. Исследуйте влияние аргумента perplexity на результат работы алгоритма на многомерных данных.
9. Визуализируйте выбранный датасет при помощи разных методов понижения размерности.
10. Сравните все три алгоритма классификации по метрике доли объясненной дисперсии. Выберите для каждого метода оптимальное количество кластеров по методу локтя.
11. Повторите измерение метрики, но уже после разбиения выборки на тестовую и обучающую. Сравните долю объясненной дисперсии на тестовой выборке.

Контрольные вопросы

1. В чем состоит задача понижения размерности? В каких случаях следует понижать размерность датасета?
2. В чем существенное различие изученных трех методов понижения размерности: PCA, LDA, t-SNE? В каких случаях какие следует применять?
3. Что такое главная компонента? Какой предметный смысл она несет?
4. В чем состоит компромисс при выборе количества компонент? Как работает для этой задачи метод локтя?
5. Какой смысл и механизм действия у аргумента perplexity в методе понижения размерности t-SNE?

Дополнительные задания

1. Изучите и примените к тому же набору данных метод независимых компонент (ICA). В чем его отличие от метода главных компонент?
2. Изучите и примените к тому же набору данных метод UMAP. В чем его специфика и отличие от других методов понижения размерности?
3. Изучите влияние понижения размерности на качество классификации. Обучите простой метод классификации на изначальном наборе данных. Сравните его точность до и после понижения размерности.
4. Постройте на примере какого-либо метода понижения размерности зависимость точности классификатора от количества компонент. Какой вывод можно сделать? Сопоставьте это график с графиком для выбора количества компонент.